

CLEAN CODE

Escrevendo código simples, legível e fácil de manter

Conceitos essenciais, boas práticas, SOLID e refatoração prática em C#.

NAMES

FUNCTIONS

CLASSES

TESTS

SOLID

O que vamos aprender?

A aula gira em torno de uma pergunta: como escrever código que outra pessoa consegue manter?

- Por que código difícil de manter custa caro
- O que é Clean Code e de onde veio essa preocupação
- Programação procedural x orientação a objetos
- Os principais princípios de código limpo
- Code Smells: como reconhecer problemas
- Como Clean Code se conecta com SOLID
- Dois exemplos completos de código C# antes e depois
- Checklist prático para aplicar no dia a dia

O problema antes do Clean Code

O software cresceu - e o custo de entender o código cresceu junto.

- No início de muitos projetos, a prioridade é fazer funcionar.
- Com o tempo chegam novas regras, pessoas, integrações e correções.
- Código sem organização começa a acumular condicionais, duplicações e dependências.
- Uma alteração pequena pode quebrar uma parte que aparentemente não tinha relação.
- O problema deixa de ser escrever código e passa a ser mudar código com segurança.

Projeto começa

Código cresce

Complexidade cresce

Manutenção fica cara

De onde vem a ideia de Clean Code?

Não nasceu de uma única regra: veio da evolução das práticas de desenvolvimento.

1970s+

**Programação
estruturada**

organizar melhor o fluxo

1980s/90s

**Orientação a
Objetos**

encapsular
responsabilidades

1990s/2000s

Refactoring + Agile

melhorar continuamente

2008

Clean Code

práticas reunidas por Robert
C. Martin

O livro Clean Code popularizou um vocabulário simples para discutir legibilidade, responsabilidade e manutenção.

O que é Clean Code?

Código limpo é código cuja intenção é fácil de entender.

LEGÍVEL

entendo rápido

SIMPLES

pouca surpresa

COESO

cada parte tem
foco

TESTÁVEL

consigo validar

Clean Code não significa código perfeito. Significa código que reduz o esforço necessário para entender e alterar o sistema.

Código que funciona x código sustentável

Os dois podem entregar o mesmo resultado hoje. Amanhã a diferença aparece.

FUNCIONA

- Entrega o resultado
- Pode concentrar regras
- Pode ser difícil de testar
- Mudanças ficam arriscadas

SUSTENTÁVEL

- Entrega o resultado
- Comunica intenção
- Separa responsabilidades
- Facilita testes e mudanças

Procedural x Orientação a Objetos

Clean Code pode existir nos dois estilos.

PROCEDURAL

```
decimal CalculateTotal(List<Item> items)
{
    var total = 0m;
    foreach (var item in items)
        total += item.Price * item.Quantity;
    return total;
}
```

ORIENTADO A OBJETOS

```
public sealed class Order
{
    public List<Item> Items { get; } = [];

    public decimal Total() =>
        Items.Sum(i => i.Price * i.Quantity);
}
```

Procedural organiza operações. OO organiza estado + comportamento. O importante é manter intenção, coesão e baixo acoplamento.

Por que OO combina bem com Clean Code?

Objetos permitem aproximar o código da linguagem do negócio.

- Encapsulamento: o objeto protege seu próprio estado.
- Responsabilidade: uma classe representa um papel claro.
- Polimorfismo: comportamentos podem variar sem grandes cadeias de if.
- Composição: pequenas partes colaboram para formar comportamentos maiores.
- Mas OO não garante Clean Code: uma classe de 2.000 linhas continua sendo um problema.

Código próximo da linguagem do negócio

```
order.AddItem(product, quantity);  
order.ApplyCoupon(coupon);  
var total = order.Total();
```

```
await payment.ChargeAsync(total);  
order.MarkAsPaid();
```


Princípio 1 - Bons nomes

Nomes são a documentação mais próxima do código.

ANTES

```
var x = 30;  
var d = GetData();  
if (u.S == 1) { ... }
```

DEPOIS

```
var paymentTimeoutSeconds = 30;  
var activeCustomers = GetActiveCustomers();  
if (user.IsActive) { ... }
```

Um nome bom responde: o que é isso? por que existe? qual intenção representa?

Princípio 2 - Funções pequenas e focadas

Uma função deve ter uma responsabilidade fácil de explicar.

FLUXO LEGÍVEL

```
public async Task CheckoutAsync(Order order)
{
    Validate(order);
    var total = CalculateTotal(order);
    await ChargeAsync(order, total);
    await SaveAsync(order);
    await NotifyCustomerAsync(order);
}
```

- Evite métodos gigantes.
- Evite misturar regra de negócio com detalhes técnicos.
- Poucos parâmetros costumam facilitar leitura e testes.
- Extraia uma função quando uma parte do código possui um nome útil.

Princípio 3 - Uma classe, uma responsabilidade

Classes gigantes viram pontos de acoplamento.

OrderService

- Calcula preço
- Salva no banco
- Envia e-mail
- Gera PDF
- Chama pagamento
- Valida cupom

Responsabilidades separadas

- PricingService
- OrderRepository
- NotificationService
- InvoiceGenerator
- PaymentGateway

Princípio 4 - Evite números e strings mágicas

A regra deve ter um nome.

ANTES

```
if (customer.Type == "VIP")  
    total *= 0.9m;  
  
if (days > 30)  
    status = 3;
```

DEPOIS

```
const decimal VipDiscount = 0.10m;  
  
if (customer.IsVip)  
    total = total.ApplyDiscount(VipDiscount);  
  
if (days > PaymentExpirationDays)  
    status = PaymentStatus.Expired;
```

Princípio 5 - Comentários com propósito

O melhor comentário explica contexto que o código não consegue expressar.

COMENTÁRIO DESNECESSÁRIO

```
// Soma 1  
count++;  
  
// Se o usuário estiver ativo  
if (user.IsActive) ...
```

CONTEXTO ÚTIL

```
// Regra contratual: pedidos podem ser  
// cancelados sem taxa por até 24 horas.  
const int FreeCancellationHours = 24;
```

Se você precisa comentar cada linha para explicar o que ela faz, provavelmente os nomes ou a estrutura podem melhorar.

Princípio 6 - Trate erros de forma clara

Falhar bem também é Clean Code.

EXEMPLO

```
try
{
    await _payment.ChargeAsync(total);
}
catch (PaymentDeclinedException ex)
{
    _logger.LogWarning(ex,
        "Payment declined for order {OrderId}", order.Id);

    return CheckoutResult.PaymentDeclined();
}
```

- Não esconda exceções.
- Não retorne false para qualquer tipo de erro.
- Dê significado às falhas.
- Inclua contexto útil em logs.
- Evite expor dados sensíveis.

Princípio 7 - Evite duplicação de regra

Duplicar conhecimento cria versões diferentes da verdade.

Exemplo

- 10% de desconto VIP calculado no checkout.
- A mesma regra copiada no relatório.
- Outra cópia no endpoint de simulação.
- Uma mudança para 15% exige lembrar de três lugares.

Melhor

- A regra tem um único responsável.
- Checkout e relatório reutilizam o mesmo conceito.
- Mudança é localizada.
- Testes protegem a regra.

Code Smells

Sinais de que o código pode estar ficando difícil de manter.

Método gigante

muita coisa para entender

Classe gigante

muitos motivos de mudança

Muitos ifs

variações misturadas

Muitos parâmetros

responsabilidade confusa

Código duplicado

regra repetida

Magic values

regra sem nome

Dependência direta

difícil substituir/testar

Nomes genéricos

intenção escondida

Refatoração

Melhorar a estrutura sem alterar o comportamento esperado.

1

Entenda

qual comportamento
precisa permanecer

2

Proteja

crie ou confirme testes

3

Melhore

nomes, funções e
responsabilidades

4

Valide

execute testes novamente

Refatorar não é reescrever tudo. É fazer pequenas melhorias seguras continuamente.

Clean Code + SOLID

Clean Code melhora a leitura. SOLID ajuda a organizar responsabilidades e dependências.

S

**Single
Responsibility**

uma responsabilidade
principal

O

Open/Closed

estender sem quebrar

L

Liskov

respeitar contratos

I

**Interface
Segregation**

interfaces focadas

D

**Dependency
Inversion**

depender de abstrações

SOLID sem complicação

Use SOLID para resolver problemas reais - não para criar camadas por obrigação.

- S: esta classe está fazendo coisas que mudam por motivos diferentes?
- O: adicionar um novo comportamento exige alterar vários ifs existentes?
- L: uma implementação pode substituir outra sem surpreender o chamador?
- I: a interface obriga alguém a implementar métodos que não precisa?
- D: minha regra de negócio depende diretamente de banco, HTTP, e-mail ou framework?

Exemplo 1 - Código difícil de manter

Um checkout que mistura regra de negócio, banco, HTTP e e-mail.

ANTES

```
public async Task<bool> Process(Order o)
{
    if (o == null || o.Items.Count == 0) return false;
    decimal t = 0;
    foreach (var i in o.Items)
        t += i.Price * i.Qty;

    if (o.Customer.Type == "VIP") t *= 0.9m;

    using var cn = new SqlConnection(_cs);
    await cn.ExecuteNonQuery("insert into Orders ...", o);

    var client = new HttpClient();
    var response = await client.PostAsJsonAsync(
        _paymentUrl, new { value = t });

    if (!response.IsSuccessStatusCode) return false;
    await _smtp.SendAsync(o.Customer.Email, "Order ok");
    return true;
}
```

- Process não explica intenção
- Regra VIP dentro do fluxo
- SQL no caso de uso
- HTTP criado diretamente
- bool esconde a falha
- Difícil testar isoladamente

Exemplo 1 - O que vamos separar?

Primeiro identificamos responsabilidades.

CheckoutService

orquestra

PricingService

calcula preço

PaymentGateway

cobra

OrderRepository

persiste

NotificationService

notifica

A classe principal fica responsável pelo fluxo; os detalhes ficam em componentes especializados.

Exemplo 1 - Depois do Clean Code

Agora o código conta a história do checkout.

DEPOIS

```
public async Task<CheckoutResult> CheckoutAsync(
    Order order, CancellationToken ct)
{
    var validation = _validator.Validate(order);
    if (!validation.IsValid)
        return CheckoutResult.Invalid();

    var total = _pricing.Calculate(order);

    var payment = await _payments.ChargeAsync(
        order.Payment, total, ct);

    if (!payment.Succeeded)
        return CheckoutResult.PaymentFailed();

    await _orders.SaveAsync(order, ct);
    await _notifications.SendConfirmationAsync(order, ct);

    return CheckoutResult.Success(order.Id);
}
```

- Nomes claros
- Fluxo legível
- Responsabilidades separadas
- Dependências substituíveis
- Erros com significado
- Muito mais fácil de testar

Exemplo 1 - Ganhos

Clean Code reduz o custo da próxima mudança.

Novo pagamento

alterar Process

novo gateway

Teste de falha

depende de HTTP

mock simples

Trocar banco

mexe no checkout

troca repositório

Entender regra

lê detalhes técnicos

lê o fluxo

ANTES

DEPOIS

Exemplo 2 - Muitos ifs e flags

O método muda completamente de comportamento dependendo dos parâmetros.

ANTES

```
public byte[] Generate(  
    List<Sale> sales,  
    bool csv,  
    bool includeTax,  
    string country)  
{  
    if (includeTax) { /* imposto */ }  
  
    if (country == "BR") { /* regra BR */ }  
    else if (country == "US") { /* regra US */ }  
  
    if (csv) { /* CSV */ }  
    else { /* PDF */ }  
  
    return data;  
}
```

- Cada flag cria caminhos diferentes.
- Novo país adiciona outro if.
- Novo formato adiciona outro if.
- Testes precisam combinar vários cenários.
- Uma função conhece responsabilidades diferentes.

Exemplo 2 - Dê nomes às variações

Interfaces simples deixam explícito o que pode variar.

CONTRATOS

```
public interface ITaxPolicy
{
    Money Calculate(Sale sale);
}

public interface IReportRenderer
{
    byte[] Render(Report report);
}
```

DEPOIS

```
public byte[] Generate(ReportRequest request)
{
    var report = _builder.Build(
        request.Sales, _taxPolicy);

    return _renderer.Render(report);
}
```

BrazilTaxPolicy, UsaTaxPolicy, CsvRenderer e PdfRenderer podem evoluir separadamente.

Exemplo 2 - Onde entrou SOLID?

O exemplo ficou mais simples porque as responsabilidades ficaram explícitas.

- SRP: cálculo fiscal e renderização não estão no mesmo componente.
- OCP: novo país ou formato pode ser adicionado por uma nova implementação.
- ISP: cada interface possui poucos métodos e um objetivo claro.
- DIP: Generate depende de ITaxPolicy e IReportRenderer, não das implementações concretas.

SOLID funciona melhor quando nasce de uma necessidade de mudança observável.

Testes e Clean Code

Testes permitem melhorar o design sem medo.

TESTE

define comportamento

REFACTOR

melhora estrutura

TESTE

confirma comportamento

Sem testes, código difícil tende a permanecer difícil porque ninguém quer arriscar quebrá-lo.

O que Clean Code NÃO é

Boas práticas também precisam de bom senso.

- Não é transformar cada linha em um método.
- Não é criar interface para toda classe.
- Não é usar Design Pattern em todo problema.
- Não é buscar o menor número possível de linhas.
- Não é seguir uma regra cegamente mesmo quando ela piora a leitura.
- Não é reescrever um sistema inteiro só porque o código antigo não está bonito.

Como aplicar no dia a dia?

Melhore o código aos poucos.

1

Leia

entenda antes de alterar

2

Renomeie

revele intenção

3

Extraia

separe
responsabilidades

4

Teste

proteja comportamento

5

Repita

deixe melhor do que
encontrou

Checklist para um Pull Request

Perguntas simples antes de considerar o código pronto.

- Os nomes deixam clara a intenção?
- Existe algum método grande demais?
- A classe possui mais de uma responsabilidade?
- Há regra duplicada?
- Existem strings ou números mágicos?
- Os erros estão claros?
- As dependências externas estão explícitas?
- Consigo testar a regra sem depender de banco ou HTTP?
- SOLID está simplificando ou apenas adicionando complexidade?
- A próxima pessoa conseguirá entender isso rapidamente?

Sugestão de prática ao vivo

Pegue o Exemplo 1 e refatore junto com a turma.

1 Identificar problemas

2 Melhorar nomes

3 Extrair cálculo

4 Extrair pagamento

5 Extrair persistência

6 Criar um teste

A turma acompanha a transformação e percebe que o ganho aparece na leitura e na facilidade de mudança.

Resumo da aula

Se você lembrar destes pontos, já consegue melhorar muito código legado.

- Código é lido muito mais vezes do que é escrito.
- Nomes devem revelar intenção.
- Funções e classes precisam de foco.
- Evite duplicação e valores mágicos.
- Erros devem ter significado.
- Separe regra de negócio de infraestrutura.
- Use SOLID para controlar responsabilidades e dependências.
- Refatore em passos pequenos e protegidos por testes.
- Clean Code é sobre facilitar entendimento e mudança.

Código limpo é código fácil de mudar

O objetivo não é perfeição. É sustentabilidade.

Funcionar é obrigatório.

Ser fácil de entender e evoluir é o que mantém o software saudável.

Nomes claros • responsabilidades pequenas • dependências explícitas • testes • refatoração contínua

Perguntas?

Referências para aprofundamento

Poucas referências, focadas no tema central.

- Robert C. Martin - Clean Code: A Handbook of Agile Software Craftsmanship (2008).
- Martin Fowler - Refactoring: Improving the Design of Existing Code.
- Robert C. Martin - princípios SOLID e design orientado a objetos.